Orquestração de Serviços e Integração Contínua com Docker e Portainer

Guilherme Sassi Costa, Pablo Luiz Madrid

Faculdade de Computação – Universidade Federal de Mato Grosso do Sul (UFMS)
Campo Grande – MS – Brasil

Resumo. Este trabalho apresenta a implantação de uma infraestrutura conteinerizada em uma máquina virtual Debian, utilizando Docker e Portainer para o gerenciamento visual dos serviços. O ambiente foi organizado em Stacks independentes para produtividade, controle de versão, integração contínua e registro privado de imagens. Foram implantados o Nextcloud integrado ao MariaDB, o Forgejo com seu Runner e o Harbor. Um projeto de calculadora em C foi enviado ao Forgejo e submetido a uma pipeline automatizada de compilação e testes. Por fim, as imagens do MariaDB, Nextcloud, Forgejo e Runner foram enviadas a um projeto privado no Harbor. Os resultados demonstraram o funcionamento integrado dos serviços e evidenciaram dificuldades relacionadas à inicialização do Portainer, divergência entre branches Git, checkout do código na pipeline e publicação de imagens no registro privado.

Abstract. This paper presents the deployment of a containerized infrastructure on a Debian virtual machine, using Docker and Portainer for visual service management. The environment was organized into independent stacks for productivity, version control, continuous integration, and private image registry. Nextcloud integrated with MariaDB, Forgejo with its Runner, and Harbor were deployed. A C calculator project was pushed to Forgejo and submitted to an automated compilation and testing pipeline. Finally, the MariaDB, Nextcloud, Forgejo, and Runner images were pushed to a private Harbor project. The results demonstrated the integrated operation of the services and highlighted difficulties related to Portainer initialization, Git branch divergence, source-code checkout in the pipeline, and image publishing to the private registry.

1. Introdução

A utilização de containers tornou-se uma prática relevante na implantação de aplicações distribuídas, pois permite isolar serviços, padronizar ambientes e simplificar processos de instalação, atualização e manutenção. O Docker possibilita empacotar aplicações e dependências em imagens reproduzíveis, enquanto o Docker Compose descreve ambientes formados por múltiplos containers.

O Portainer complementa esse ecossistema ao oferecer uma interface web para criação de Stacks, acompanhamento de containers, configuração de volumes, redes e variáveis de ambiente. Em paralelo, práticas de integração contínua e entrega contínua (CI/CD) automatizam etapas de validação, compilação e testes após alterações no código-fonte, reduzindo erros manuais.

Este trabalho teve como objetivo configurar, em uma máquina virtual Debian, uma infraestrutura conteinerizada gerenciada pelo Portainer. O ambiente foi dividido em uma Stack de produtividade com Nextcloud e MariaDB, uma Stack de desenvolvimento com Forgejo e Forgejo Runner e uma Stack de registro privado baseada no Harbor. Também foi validado um fluxo de CI para um projeto em C e o envio das imagens utilizadas para um repositório privado.

2. Ambiente e Arquitetura da Solução

A infraestrutura foi preparada em uma máquina virtual Debian executada no VirtualBox. O Docker Engine foi utilizado para execução dos containers e o Portainer Community Edition como ponto central de administração. Após a implantação inicial do Portainer, as demais operações foram realizadas por sua interface web.

Componente	Função no ambiente
Debian/VirtualBox	Debian (debian-13.5.0-amd64-netinst)
Docker Engine	Execução e isolamento dos serviços.
Portainer CE	Gerenciamento visual de Stacks, containers e volumes.
Nextcloud + MariaDB	Armazenamento de arquivos e banco de dados persistente.
Forgejo + Runner	Hospedagem Git e execução automatizada da pipeline.
Harbor	Registro privado das imagens Docker.

A comunicação interna das aplicações foi realizada pelos nomes dos serviços definidos nos arquivos Compose. Volumes Docker foram empregados para preservar bancos de dados, configurações e arquivos mesmo após reinicializações dos containers.

3. Configuração da Infraestrutura Base

3.1 Instalação do Docker e do Portainer

O Docker Engine e seus componentes foram instalados diretamente no terminal do Debian. Em seguida, o Portainer CE foi iniciado em um container com acesso ao socket do Docker e volume persistente para suas configurações. Essa foi a última implantação realizada por linha de comando.

A instalação do Docker foi realizada no Debian utilizando os pacotes oficiais disponibilizados pela Docker Foundation. Foram instalados o Docker Engine, Docker CLI, Containerd e os plugins Docker Compose e Buildx com os seguintes comandos:

```
sudo apt-get update

sudo apt-get install docker-ce docker-ce-cli \
containerd.io docker-buildx-plugin \
docker-compose-plugin
```

Assim como para a verificação do serviço do docker o comando necessário é:

docker --version

docker compose version

Após a instalação do Docker, foi criado um volume persistente para armazenar os dados do Portainer. Em seguida, o Portainer Community Edition foi implantado como container Docker, disponibilizando as portas 8000 e 9443 para acesso à interface administrativa, utilizando-se de;

sudo docker volume create portainer_data

e então;

sudo docker run -d \

-p 8000:8000 \

-p 9443:9443 \

--name portainer \

--restart=always \

-v /var/run/docker.sock:/var/run/docker.sock \

-v portainer_data:/data \

portainer/portainer-ce:latest

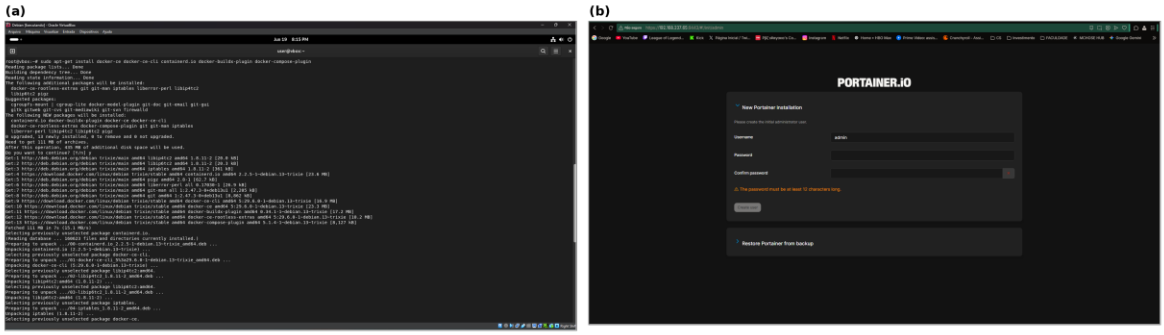


Figura 1. Instalação do Docker e criação inicial do usuário administrador no Portainer.

Durante a criação do usuário administrador, a sessão inicial expirou por segurança. O acesso foi restabelecido com a reinicialização do container do Portainer, permitindo finalizar o cadastro. Depois dessa etapa, as demais Stacks foram criadas pela interface web.

4. Stack de Produtividade: Nextcloud e MariaDB

A Stack de produtividade foi composta pelos serviços db e app. O serviço db utiliza a imagem MariaDB 10.11, armazena os dados no volume mariadb_data e recebe as credenciais por variáveis de ambiente. O serviço app utiliza a imagem do Nextcloud, publica a porta 8080 e mantém os arquivos

no volume nextcloud_data. O hostname db foi usado pelo Nextcloud para localizar o banco na rede interna da Stack. A presença dos volumes persistentes evita a perda de dados quando os containers são reiniciados ou recriados.

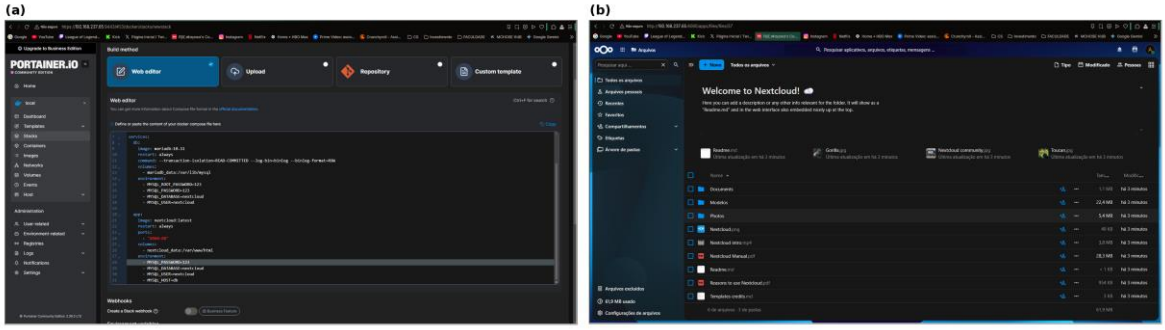


Figura 2. Detalhes da Stack de produtividade no Portainer e interface do Nextcloud em execução.

5. Controle de Versão e Integração Contínua

5.1 Forgejo e Runner

O Forgejo foi implantado como servidor privado de repositórios Git na porta 3000. Um segundo serviço executou o Forgejo Runner, com volume próprio e acesso ao socket Docker. O Runner foi inicialmente registrado por um token temporário e, após o registro, passou a executar em modo daemon.

5.2 Envio do projeto e execução da pipeline

O projeto de calculadora foi versionado e enviado ao Forgejo. A primeira tentativa de push utilizou a branch main, porém o repositório local estava na branch master. Após a correção do comando, o envio foi concluído. O workflow foi então ajustado para operar de acordo com o ambiente configurado no servidor.

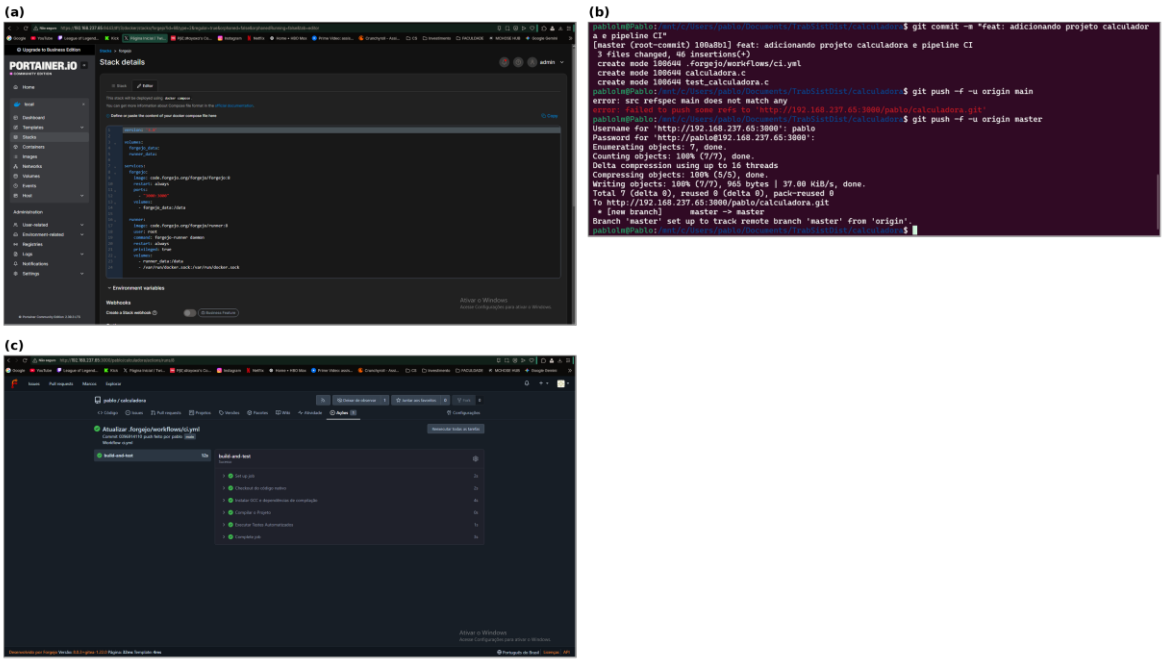


Figura 3. Stack de desenvolvimento, envio do projeto da calculadora e acionamento do workflow no Forgejo.

A pipeline fornecida originalmente utilizava actions/checkout@v4. No ambiente local, o checkout nativo não foi executado corretamente pelo Runner. A solução foi substituir essa etapa por um comando wget que baixa o repositório compactado do Forgejo e o extrai antes da compilação. Segundo recomendação, inicialmente utilizaríamos o Alpine(distribuição Linux independente), porém, ele não tem JavaScript nativamente, ou seja, ele não consegue trabalhar com o node diretamente, então, optamos por essa opção assim como conversado justamente para que tudo continuasse funcionando corretamente. Em seguida, o job instala o GCC no container do Runner, compila os arquivos calculadora.c e test_calculadora.c e executa os testes.

6. Registro Privado de Imagens com Harbor

O Harbor foi obtido e configurado para fornecer um registro privado na rede local. Sua Stack reúne serviços internos como registry, registryctl, PostgreSQL, Redis, core, portal, jobservice, proxy e log. Após a criação do projeto privado projetos, foi realizado login no registro e, em seguida, as imagens MariaDB, Nextcloud, Forgejo e Forgejo Runner receberam tags apontando para o endereço local do Harbor.

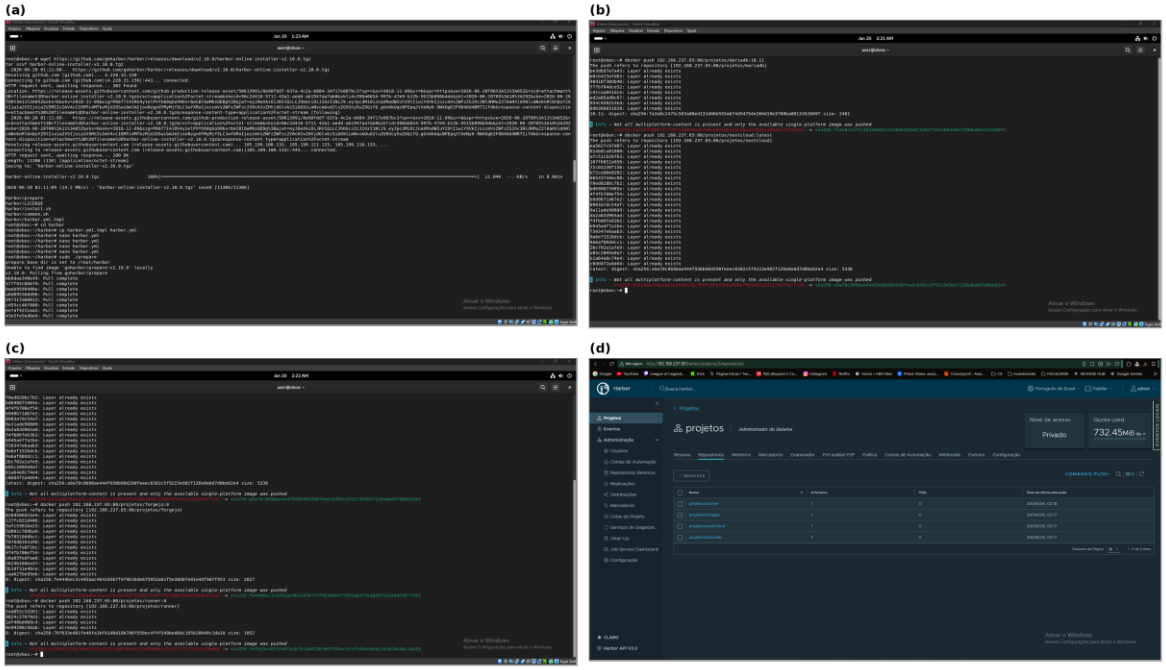


Figura 4. Push das imagens para o Harbor e confirmação dos repositórios no projeto privado.

A interface do Harbor confirmou a presença dos quatro repositórios exigidos. O resultado evidencia o funcionamento do registro privado e a publicação bem-sucedida das imagens utilizadas nas demais Stacks do ambiente.

7. Declaração das Stacks e do Workflow

Os arquivos completos acompanham o relatório no arquivo ZIP de entrega. Como os arquivos do Nextcloud/MariaDB, Forgejo/Runner e workflow possuem tamanho reduzido, eles podem ser apresentados no corpo do artigo. Já o Compose do Harbor possui aproximadamente 255 linhas e diversos componentes gerados automaticamente; por isso, sua estrutura é resumida no texto e o arquivo integral permanece apenas no ZIP.

Código 1. Docker Compose da Stack Nextcloud/MariaDB.

```
version: '3.8'

volumes:
  mariadb_data:
  nextcloud_data:

services:
  db:
    image: mariadb:10.11
    restart: always
    command: --transaction-isolation=READ-COMMITTED --log-bin=binlog --binlog-
format=ROW
    volumes:
      - mariadb_data:/var/lib/mysql
    environment:
      - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}
      - MYSQL_PASSWORD=${MYSQL_PASSWORD}
      - MYSQL_DATABASE=nextcloud
      - MYSQL_USER=nextcloud

  app:
    image: nextcloud:latest
    restart: always
    ports:
      - "8080:80"
    volumes:
      - nextcloud_data:/var/www/html
    environment:
      - MYSQL_PASSWORD=${MYSQL_PASSWORD}
      - MYSQL_DATABASE=nextcloud
      - MYSQL_USER=nextcloud
      - MYSQL_HOST=db
```

Código 2. Docker Compose da Stack Forgejo/Runner.

```
version: '3.8'

volumes:
  forgejo_data:
  runner_data:

services:
  forgejo:
    image: code.forgejo.org/forgejo/forgejo:8
    restart: always
    ports:
      - "3000:3000"
    volumes:
      - forgejo_data:/data

  runner:
    image: code.forgejo.org/forgejo/runner:8
    user: root
    command: forgejo-runner daemon
```

```
    restart: always
    privileged: true
    volumes:
      - runner_data:/data
      - /var/run/docker.sock:/var/run/docker.sock
```

Código 3. Workflow de integração contínua da calculadora.

```
name: CI de Testes em C (Alpine)

on:
  push:
    branches: [ "main" ]

jobs:
  build-and-test:
    runs-on: alpine

    steps:
      - name: Checkout do código nativo
        run: wget -qO- http://192.168.237.65:3000/pablo/calculadora/archive/main.tar.gz | tar xvz --
strip-components=1

      - name: Instalar GCC e dependências de compilação
        run: |
          apk update
          apk add --no-cache gcc musl-dev

      - name: Compilar o Projeto
        run: gcc calculadora.c test  calculadora.c -o suite  de  testes

      - name: Executar Testes Automatizados
        run: ./suite_de_testes
```

7.1 Estrutura resumida do Harbor

Serviço	Responsabilidade
log	Centralização e rotação de logs.
registry	Armazenamento e distribuição das imagens.
registryctl	Controle das operações do registro.
postgresql	Persistência dos dados internos.
core	Lógica central e API do Harbor.
portal	Interface web de administração.
jobservice	Execução de tarefas assíncronas.
redis	Cache e filas internas.
proxy	Ponto de entrada HTTP/HTTPS.

O arquivo integral docker-compose yml HARBOR.yml foi mantido no pacote de entrega, preservando volumes, dependências, redes, arquivos de configuração e políticas de reinicialização necessárias ao funcionamento da plataforma.

8. Análise e Diagnóstico de Falhas

Falha observada	Causa identificada	Solução aplicada
Expiração da configuração inicial do Portainer.	Tempo excedido antes da criação do administrador.	Reinicialização do container e conclusão imediata do cadastro.
Falha no primeiro git push.	Branch local master diferente da branch main informada.	Envio para a branch existente e posterior alinhamento do workflow.
Checkout da pipeline não executava no Runner.	Incompatibilidade da action de checkout no ambiente configurado.	Substituição por wget e extração do arquivo main.tar.gz.
Runner exigia token somente no registro inicial.	O comando de registro não deve permanecer no serviço definitivo.	Registro temporário e alteração posterior para forgejo-runner daemon.
Grande número de serviços no Harbor.	Arquitetura composta por registro, banco, cache, proxy e serviços auxiliares.	Validação pelo Portainer até todos os containers permanecerem running ou healthy.

9. Resultados

Ao final da implementação, o Portainer apresentou as Stacks forgejo, harbor e produtividade, além dos containers associados. O Nextcloud permaneceu acessível com autenticação, o Forgejo armazenou o projeto e executou a pipeline com sucesso, e o Harbor recebeu as quatro imagens solicitadas.

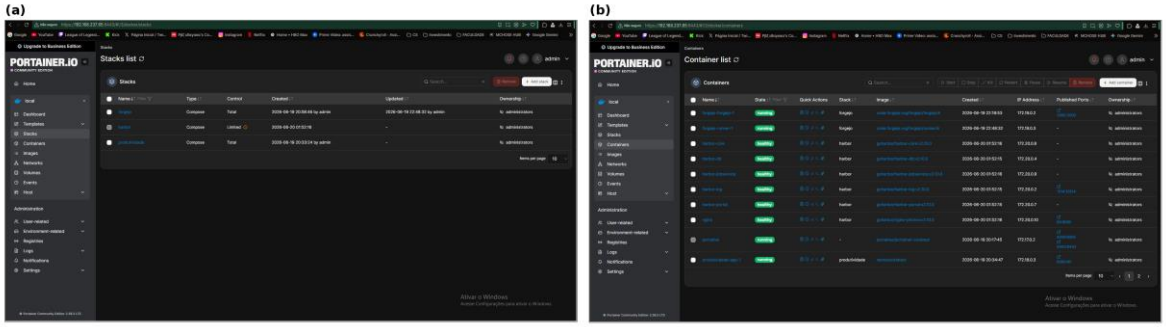


Figura 5. Visão final das Stacks e dos containers ativos no Portainer.

Os resultados confirmam a integração entre armazenamento persistente, controle de versão, automação de testes e registro privado de imagens. A separação em Stacks facilitou a organização e o diagnóstico, enquanto o Portainer permitiu acompanhar o estado dos componentes em uma interface única.

10. Conclusão

A implantação demonstrou como Docker, Portainer e ferramentas DevOps podem ser combinados em uma infraestrutura modular. O Nextcloud e o MariaDB atenderam ao cenário de produtividade com persistência; o Forgejo e o Runner permitiram versionar o código e executar testes automaticamente; e o Harbor centralizou as imagens utilizadas em um projeto privado.

As falhas encontradas durante o processo foram úteis para compreender aspectos práticos de inicialização segura, branches Git, registro de agentes, compatibilidade de workflows e operação de um registro privado.

Referências

DOCKER. Docker Documentation. Disponível em: <https://docs.docker.com/>.

PORTAINER. Portainer Documentation. Disponível em: <https://docs.portainer.io/>.

NEXTCLOUD. Nextcloud Administration Manual. Disponível em: <https://docs.nextcloud.com/>.

FORGEJO. Forgejo Documentation. Disponível em: <https://forgejo.org/docs/>.

HARBOR. Harbor Documentation. Disponível em: <https://goharbor.io/docs/>.